

**Baptista Szeretetszolgálat Ejsz Széchenyi István
Gimnáziuma és Technikuma**

Vizsgaremek

Bookr

Szoftverfejlesztő és -tesztelő

Készítették:

Vasvári Zoltán Benjámin

Sándor László

Ujhelyi Hunor

Pécs

2026

Tartalomjegyzék

Feladat rövid ismertetése	4
Használt technológiák	4
Programozási nyelvek	4
Egyéb technológiák	5
Adatbázis	5
Adatbázis rövid magyarázata	5
Triggerek	7
Fejlesztés módszertan	7
Scrum	7
Jira	7
Git / GitHub	7
Elvégzett tesztek	8
Tesztelési fajták	8
Teszteredmények	8
Oldal működésének részletes leírása	8
Vendég	8
Ügyfél (user)	8
Szakember (staff)	9
Szalontulajdonos (owner)	9
Superadmin	9
Fejlesztői dokumentáció	10
Fejlesztési technológiák	10
HTTP protokollok	10
A projektben való rétegek felépítése	11
Config package	11
Controller package	11
DTO package	11
Filter package	11
Model package	12
Security package	12
Service package	12
Servlet package	12
Util package	12
A frontend részletes leírása	13
Könyvtárstruktúra	13

Komponensek	13
Szolgáltatások és core elemek	13
Csapatmunka.....	15
Szerepek.....	15
Jövőbeli terveink.....	16

Feladat rövid ismertetése

A Bookr egy webes időpontfoglaló alkalmazás, amit szépségszalonok és ügyfelek számára fejlesztettünk. Az ötlet onnan jött, hogy a telefonos foglalás ma már elavult – sokszor kényelmetlenebb, mint kellene, és mind az ügyfelek, mind a szalonok részéről felesleges többletmunkával jár. Ezért egy modern, könnyen használható platformmal váltottuk ki, ahol az egész folyamat pár kattintásból áll.

A rendszer célja, hogy a szépségiparban dolgozók és az ügyfelek között egyszerűbb legyen az időpontok szervezése. A szalonok oldaláról nézve rengeteg kis adminisztrációs feladat halmozódik fel nap mint nap – ki foglalt, mikor, kinél, mi lett lemondva. A Bookr ezeket egy helyen kezeli, átláthatóan.

Az alkalmazás három különböző felhasználói csoportnak szól:

- **Ügyfeleknek** – bármikor tudnak időpontot foglalni, látják a szabad és foglalt időpontokat, és maguk választják ki a szakembert.
- **Szalonoknak** – átlátható foglaláskezelés, automatikus e-mail értesítések és bevételi statisztikák.
- **Szakembereknek** – látják a saját naptárjukat, kezelhetik a munkaidejüket, és könnyebben kommunikálhatnak az ügyfelekkel.

A foglalási folyamat röviden: regisztráció, szalon és szolgáltatás kiválasztása, időpont lefoglalása – majd a látogatás után értékelés.

Használt technológiák

Programozási nyelvek

1. Frontend
 - HTML
 - CSS
 - TypeScript
 - Angular (keretrendszer)
2. Backend
 - Java

Egyéb technológiák

1. **Git** – Elosztott verziókezelő rendszer, amely lehetővé teszi a párhuzamos fejlesztést és a kódbázis nyomon követését a csapaton belül.
2. **Jira** – Projektmenedzsment eszköz, amellyel a feladatokat, határidőket és a fejlesztési folyamatokat szervezzük és követjük nyomon.
3. **Figma** – Webes UI tervező és prototípus-készítő eszköz, amellyel a felhasználói felület kinézetét terveztük meg a fejlesztés megkezdése előtt.
4. **Docker** – Konténerizációs technológia, amellyel az alkalmazás éles környezetben, Ubuntu szerveren fut.
5. **Cloudflare** – Hálózati forgalom védelmét és a domain kezelést biztosítja.
6. **WildFly** – Java alkalmazásszerver, amely a session- és kapcsolatkezelést végzi.

Adatbázis

Adatbázis típusa

Az adatbázisunk MySQL alapokon működő relációs adatbázis. A relációs adatbázis strukturált és hatékony adatkezelést biztosít, tárolt eljárások segítségével optimalizált lekérdezésekkel.

Adatbázis rövid magyarázata

Az adatokat MySQL adatbázisban tároljuk. A tervezés során nagy figyelmet fordítottunk arra, hogy az adatstruktúra logikus és könnyen bővíthető legyen. Az adatbázis összesen **22** táblából épül fel, lefedve a szalonkezelés, foglalások, felhasználók és értesítések teljes körét.

A legfontosabb táblák:

- **users** – a regisztrált felhasználók adatait tárolja – név, email, jelszó, 2FA beállítások.
- **companies** – a szalonok adatait tartja nyilván – cím, nyitvatartás, foglalási szabályok.
- **staff** – a szalonhoz rendelt szakemberek adatait tárolja.
- **appointments** – itt vannak a foglalások – státusszal, időponttal, árral és megjegyzésekkel együtt.
- **services** – a szalon kínálatában szereplő szolgáltatások adatai.
- **opening_hours** / **staff_working_hours** – a szalon nyitvatartása és az egyes szakemberek saját munkaideje.
- **reviews** – a leadott értékelések táblája.

- Az alábbi diagrammon látható az adatbázis és annak kapcsolatainak felépítése:



Triggerek

Néhány műveletet triggerrel automatizáltunk:

- **after_staff_insert** – új szakember rögzítésekor fut le automatikusan.
- **after_staff_update_audit** – ha változnak egy szakember adatai, automatikusan naplózódik.
- **after_user_update_audit** – felhasználói adatok változásakor szintén naplóbejegyzés keletkezik.
- **users_before_insert_guid / users_before_update_guid** – minden felhasználóhoz automatikusan generálódik egy egyedi GUID azonosító.

Fejlesztés módszertan

Scrum

Agilis, Scrum-alapú módszertannal dolgoztunk. A fejlesztést sprintekre osztottuk, így mindig volt egy konkrét cél, amit az adott időszakban el kellett érni. Ha valami változott vagy nem úgy sült el, ahogy terveztük, könnyen tudtunk irányt váltani.

Jira

Feladatkezelésre Jirát használtunk, tábla nézetben. A kártyákat az éppen aktuális státuszuk szerint mozgattuk az oszlopok között: Icebox (alacsony prioritású, de elvégezhető feladatok), To Do, In Progress, In Review és Done. Így mindenki látta, hogy éppen mi van folyamatban és mi vár még rá.

Git / GitHub

Verziókezelésre Gitet és GitHubot használtunk. Mindenki a saját branchén dolgozott, és szükség szerint mergeltük be a változtatásokat a main ágba. Így elkerültük, hogy egymás munkájába beleszaladjunk, és a főág is mindig működőképes maradt.

Elvégzett tesztek

Tesztelési fajták

Három területen végeztünk tesztelést: a backend API-n, az adatbázis stored procedure-ökön és a frontenden. Minden tesztesetet Google Táblázatokban dokumentáltunk – rögzítettük a célt, a lépéseket, az elvárt és a tényleges eredményt, és hogy átment-e vagy sem.

1. Backend API tesztelés – Postmannel teszteltük a REST API végpontokat. Összesen 67 teszteset született, amelyek lefedik a regisztrációt, bejelentkezést, foglaláskezelést, képfeltöltést és a dashboard funkciókat.
2. Adatbázis tesztelés – A stored procedure-öket közvetlenül az adatbázison teszteltük. 51 teszteset készült, amelyek az időpontok teljes életciklusát lefedik a létrehozástól a teljesítésig, de a munkaidő- és kivételkezelést is tartalmazzák.
3. Frontend manuális tesztelés – A felületet böngészőben, kézzel teszteltük végig. 17 teszteset készült, amelyek a regisztrációtól a profilkezelésen és a kedvenceken át az adminisztrációs paneig mindent érintenek.

Teszteredmények

Összesen 135 teszteset készült. A megtalált hibákat javítás után újra átfuttattuk, és a végeredmény alapján a tesztek nagy része sikeresen teljesült.

Oldal működésének részletes leírása

Vendég

Bejelentkezés nélkül is teljesen szabadon lehet böngészni a szalonok között. Láthatók a bemutatkozó szövegek, képek, nyitvatartás, szolgáltatások, szakemberek és az értékelések is. A cím térképen is megjeleníthető. A foglalási folyamatot végig lehet nézegetni – látszanak a szabad és már foglalt időpontok –, de időpontot csak bejelentkezés után lehet ténylegesen lefoglalni. Természetesen itt van lehetőség regisztrálni is.

Ügyfél (user)

Bejelentkezés után az ügyfél végig tud menni a teljes foglalási folyamaton: szalon, szolgáltatás, szakember, időpont – majd megerősítés. A profiloldalán módosíthatja a nevét, telefonszámát és e-mail címét – utóbbihoz hitelesítő e-mail megy ki. Profilképet is feltölthet, kedvenc szalonokat jelölhet meg, és látja az összes korábbi és aktuális foglalását. A már teljesített időpontokra értékelést írhat. Az értesítési preferenciák és a dark mód is

személyre szabható. Ha valaki biztonságosabb hozzáférést szeretne, bekapcsolhatja a kétfaktoros hitelesítést – ez Google Authenticatorral vagy hasonló applikációval működik.

Szakember (staff)

A szakember normál felhasználóként is tud foglalni – akár más szalonba, akár a sajátjukba. A saját dashboardján látja a mai napját: hány foglalása van, mik a következők, milyen szolgáltatásai aktívak és mikor dolgozik. Naptár nézetben áttekintheti az összes időpontját, a mai napot pedig egy idővonalyszerű nézetben is látja. A munkaideje testre szabható – ez eltérhet a cég nyitvatartásától. Kiválaszthatja, hogy a szalon kínálatából mit szeretne vállalni. A foglalásokat lemondhatja vagy módosíthatja, és az ügyfelek elérhetőségéhez is hozzáfér.

Szalontulajdonos (owner)

A tulajdonosnak van a legtöbb jogosultsága. A dashboardon azonnal látja a legfontosabb számokat: mai ügyfélszám, heti bevétel, napi átlag, következő időpontok és a legújabb értékelések. A naptárban az összes alkalmazott időpontja megjelenik, szakember szerint szűrve. A csapat fülön áttekintheti a dolgozóit és a következő foglalásaikat. Új munkatársat e-mail cím megadásával hívhat meg – ha az illető már regisztrált, egyszerűbb a folyamat, ha nem, akkor e-mailt kap a meghívásról. A meghívás egy hétig érvényes, és vissza is vonható. Az ügyfelek fülön láthatók azok, akik valaha foglaltak a szalonnál. A szolgáltatások fülön kategóriákat és szolgáltatásokat hozhat létre vagy szerkeszthet. A pénzügyek oldalon heti, havi és éves bontásban látja a bevételeket, az értékelések pedig külön szűrhető nézetben érhetők el. A cég összes adata módosítható: képek, nyitvatartás, cím, cégnév, üzleti szabályok – és ideiglenes zárva tartás is beállítható.

Superadmin

A superadmin mindent lát és mindent kezelhet – felhasználókat, cégeket, foglalásokat. Hozzáfér az audit logokhoz, ahol nyomon követhető, hogy ki mit változtatott a rendszerben. A jövőben tervezzük azt is, hogy lejárt e-maileket (pl. regisztrációs hitelesítőt) újra lehessen küldeni innen.

Fejlesztői dokumentáció

Fejlesztési technológiák

A frontendet Angular 17+ keretrendszerrel fejlesztettük TypeScript nyelven. A backend Java JAX-RS REST API-ra épül, WildFly alkalmazáserveren fut. Az adatokat MySQL adatbázisban tároljuk.

HTTP protokollok

A frontend és a backend közötti kommunikációhoz az alábbi HTTP metódusokat használjuk:

1. GET – adatok lekérésére, például szalonlista, időpontok, felhasználói profil.
2. POST – új adatok küldésére, például regisztrációkor, foglalás leadásakor, bejelentkezésnél.
3. PUT – meglévő adatok frissítésére, például profil- vagy szalonadatok módosításakor.
4. DELETE – törléshez, például foglalás lemondásakor vagy kedvenc eltávolításakor.

A projektben való rétegek felépítése

Config package

A Config package tartalmazza az alkalmazás indításához szükséges beállításokat – a környezeti változók beolvasását és az e-mail küldés konfigurációját.

- EmailConfig – Az e-mail küldéshez szükséges konfigurációs beállításokat tartalmazza.
- EnvConfig – A környezeti változók (pl. adatbázis kapcsolati adatok, titkos kulcsok) kezelését végzi.

Controller package

A controllerek fogadják a beérkező HTTP kéréseket, kinyerik a szükséges adatokat, meghívják a megfelelő service-t, majd visszaküldik a választ. Nagyjából a 22 adatbázis táblát követik, néhány egyedi végponttal kiegészítve.

DTO package

A DTO-k (Data Transfer Object) a kliens és a szerver közötti adatcsere struktúráját határozzák meg – csak a szükséges mezők kerülnek átadásra.

- checkStaffInviteTokenDTO – a meghívó token ellenőrzéséhez.
- CompanyRegisterRequest – a cégregisztráció adatait fogja össze.
- OwnerPanelDTO – a tulajdonosi dashboard adatait adja vissza.
- staffPanelDTO – a szakember dashboard adatait adja vissza.

Filter package

A Filter package a beérkező kérések előszűrésével foglalkozik.

- CorsFilter – CORS beállítások kezelése, vagyis hogy mely forrásokból fogadunk kérést.

Model package

A modellek az adatbázis tábláit képviselik Java osztályok formájában – minden táblához tartozik egy model osztály.

Security package

A Security package a hitelesítéssel és jelszókezeléssel foglalkozik.

- JWT – a tokenek generálását és ellenőrzését végzi.
- PasswordHasher – jelszavak biztonságos hash-elése és összehasonlítása.

Service package

A service-ek tartalmazzák az üzleti logikát. Ők hívják meg a stored procedure-öket és dolgozzák fel az adatbázisból visszakapott adatokat.

Servlet package

A Servlet package speciális szerver szintű kérések kezelésére szolgál.

- ImageServlet – a feltöltött képeket szolgálja ki a frontendnek.

Util package

Az Util package-ben vannak azok a segédosztályok, amelyeket a projekt több részén is használunk.

- AesEncryptionUtil – érzékeny adatok AES titkosítása.
- EmailHtmlTemplates – a kiküldött e-mailek HTML sablonjait tárolja.
- ErrorResponseBuilder – egységes hibaüzenetek összeállítása.
- FileStorageUtil – képek és fájlok tárolásának kezelése.
- FileValidator – a feltöltött fájlok ellenőrzése (méret, típus).
- RoleChecker – ellenőrzi, hogy az adott felhasználónak van-e jogosultsága az adott műveletre.
- StoredProcedureUtil – NULL értékű paraméterek biztonságos átadása stored procedure-öknek.
- ValidationUtil – általános validációs segédfüggvények.

A frontend részletes leírása

Könyvtárstruktúra

A frontend projekt mappastruktúrája négy fő részre tagolódik:

- **core** – az alkalmazás alapjai – interceptorok, guard-ok, service-ek, modellek, pipe-ok és egyéb segédeszközök.
- **features** – a nagyobb funkcionális egységek önálló modulokban: owner-dashboard, staff-dashboard, company-registration, superadmin-dashboard.
- **shared** – több helyen is újrafelhasznált komponensek: header, footer, modal ablakok, success overlay, chat.
- **profile** – a profilhoz kapcsolódó oldalak: foglalások, beállítások, kedvencek, személyes adatok.

Komponensek

Minden oldal és funkcionalitás saját komponensbe van szervezve:

- **login-page, register-page** – Bejelentkezési és regisztrációs oldalak.
- **main-page** – Az alkalmazás főoldala.
- **search-results, sel-industry** – Szalonkereső oldal, illetve egy adott szalon részletes oldala, ahol megtekinthetők a szakemberek, szolgáltatások, értékelések és képek.
- **learnmore-page** – Bemutató oldal a rendszerről.
- **appointment, appointment-select, appointment-payment** – A foglalási folyamat lépései.
- **staff-invite-page** – Szakember meghívó oldal.
- **verify-email, reset-password** – E-mail hitelesítés és jelszó-visszaállítás.
- **profile** – Felhasználói profil, foglalások, kedvencek és beállítások kezelése.
- **owner-dashboard** – Tulajdonosi vezérlőpult: áttekintés, naptár, csapat, ügyfelek, szolgáltatások, értékelések, pénzügyek, beállítások.
- **staff-dashboard** – Szakember vezérlőpult: naptár, foglalások, munkaidő beállítás, profil, beállítások.
- **company-registration** – Lépésenkénti cégregisztrációs folyamat: céginformációk, üzleti adatok, képfeltöltés, nyitvatartás, tulajdonos adatok.
- **superadmin-dashboard** – Szuperadmin vezérlőpult: áttekintés, cégek, tulajdonosok, szakemberek, foglalások, audit logok.

Szolgáltatások és core elemek

- **Interceptors** – automatikusan csatolják a JWT tokent minden API kéréshez.
- **Guards** – megakadályozzák, hogy jogosulatlan felhasználók hozzáférjenek védett oldalakhoz.

- **Services** – az API kommunikáció és az üzleti logika kezelése.
- **Models** – TypeScript interfészek a típusbiztos adatkezeléshez.
- **Pipes** – egyedi adatformázó pipe-ok.

Csapatmunka

Szerepek

A Bookrt hárman fejlesztettük, mindenki a saját területére fókuszálva.

Vasvári Benjamin vállalt szerepei és felelősségi területei:

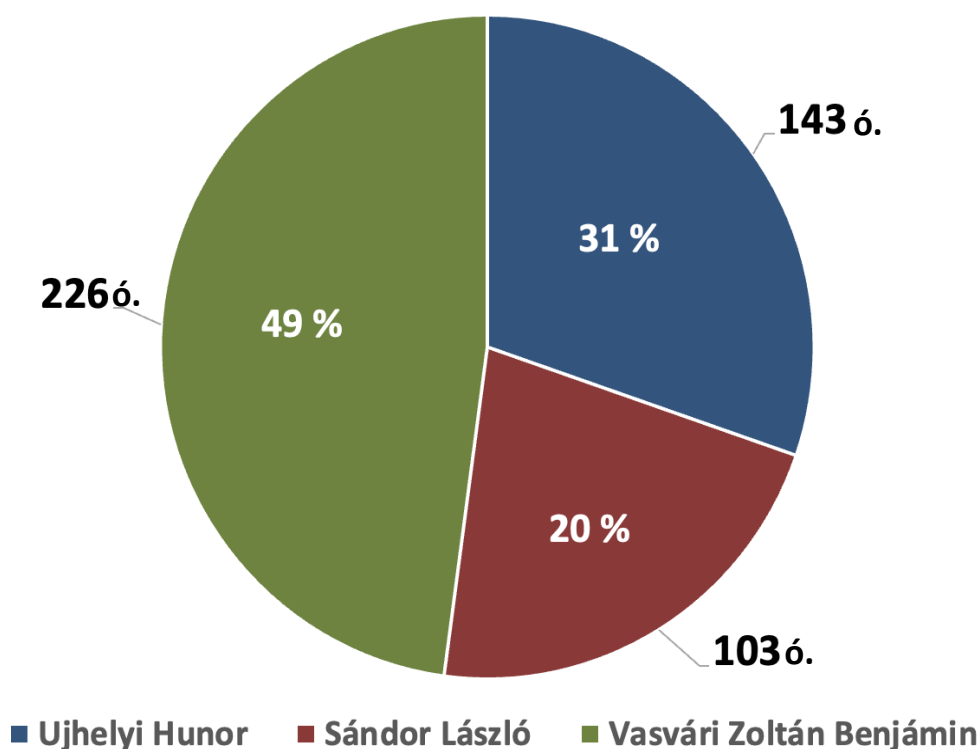
1. Project Manager – A Bookr ötlete és koncepciója hozzá köthető, ő látta a legtisztábban az egyes célokat, feladatokat. A fejlesztési folyamat koordinálása, a célok kitűzése és a csapat összehangolása mind az ő feladata volt.
2. Backend fejlesztő – A szerveroldali logika tervezése és megvalósítása Java JAX-RS technológiával. Ő írta a REST API végpontokat, implementálta a JWT alapú hitelesítést, a szerepköralapú jogosultságkezelést, és ő állította be a Docker alapú éles környezetet is.

Sándor László vállalt szerepei és felelősségi területei:

1. Frontend fejlesztő – A teljes felhasználói felület az ő munkája: Angular 17+ alapú komponensek, service-ek, guard-ok és interceptorok. Emellett a Figma-ban ő készítette el a UI terveket is, amelyek alapján aztán a fejlesztés zajlott.

Ujhelyi Hunor vállalt szerepei és felelősségi területei:

1. Adatbázis fejlesztő – A 22 táblás MySQL adatbázis megtervezése és felépítése. Ő írta a stored procedure-öket és a triggereket is, amelyek az adatbázis szintjén kezelik az automatizált műveleteket.



Jövőbeli terveink

A Bookr fejlesztése a vizsga után sem áll meg. Néhány dolog még hiányzik vagy félkész maradt, ezeket tervezzük befejezni:

1. Superadmin felület – a dashboard még nem teljesen kész. Tervezzük az audit log teljes kezelését és a lejárt e-mailek újraküldési lehetőségét.
2. Staff funkciók – a meghívási folyamat még nem teljesen működik, ezt be kell fejezni. A staff dashboard is kap még néhány kiegészítést.
3. Térképes keresés – tervezzük, hogy a szalonok térképen is megjelenjenek és kereshetők legyenek helyszín alapján.
4. Egyes funkciók még finomhangolásra szorulnak, hogy hiba nélkül, megfelelően működjenek.